

Bayesian Linear Regression examples

Arnab Hazra

First we simulate data from a multiple linear regression model and then we fit different Bayesian linear regression models to see the performances.

The model for the i -th observation would be $Y_i = \beta_0 + \sum_{i=1}^p \beta_i X_i + \varepsilon_i$ for $i = 1, \dots, n$. Here, $\varepsilon_i \sim \text{Normal}(0, \sigma^2)$.

```
rm(list = ls())

mydir <- "/Users/hazraa/Desktop/Bayesian MTH535A/Slides/Chap4_Regression"
setwd(mydir)

n <- 100
p <- 4

# obtain design matrix

X <- cbind(1, matrix(runif(n * p), n, p))

head(X)

##      [,1]      [,2]      [,3]      [,4]      [,5]
## [1,]  1 0.06841208 0.48061196 0.9642156 0.8976220
## [2,]  1 0.04852032 0.88810282 0.4690519 0.8138910
## [3,]  1 0.84579507 0.51099086 0.1069108 0.9294532
## [4,]  1 0.64867903 0.04954788 0.8132383 0.1902904
## [5,]  1 0.03491437 0.48901439 0.4563798 0.7453280
## [6,]  1 0.92588273 0.68205801 0.1797962 0.5104165

apply(X[, 2:(p+1)], 2, mean)

## [1] 0.5298455 0.5173766 0.5159642 0.5441741

apply(X[, 2:(p+1)], 2, sd)

## [1] 0.3025016 0.2970845 0.2964455 0.2765502
# center and scale covariates to have mean zero and variance one

X[, 2:(p+1)] <- apply(X[, 2:(p+1)], 2, scale)

apply(X[, 2:(p+1)], 2, mean)

## [1] -1.319084e-16 -1.071842e-16 -3.230502e-17  7.653871e-17

apply(X[, 2:(p+1)], 2, sd)

## [1] 1 1 1 1
```

```
beta.true <- c(5, rep(1, 2), rep(0, 2))
beta.true
```

```
## [1] 5 1 1 0 0
```

```
sigmaSq.true <- 0.5^2
```

```
Y <- c(X %*% beta.true) + rnorm(n, mean = 0, sd = sqrt(sigmaSq.true))
```

```
Y
```

```
## [1] 2.829250 4.777993 6.915981 3.008010 3.523476 6.547567 3.177566 5.705362
## [9] 4.000410 6.078321 3.265264 5.336629 5.363405 5.471295 3.625045 3.950470
## [17] 7.093620 7.065576 3.566668 6.543988 6.333988 5.563302 6.204888 4.213421
## [25] 5.476564 4.657128 3.302367 5.009199 7.524482 6.647091 6.882651 3.839517
## [33] 5.630522 3.799183 6.927318 6.044654 3.195049 4.494109 3.592339 4.808377
## [41] 6.568266 6.284739 4.650531 7.310014 4.636420 3.513332 2.580465 5.405046
## [49] 3.127134 3.044315 6.642667 7.043891 3.687085 5.417683 3.721315 4.002854
## [57] 6.968135 2.958439 5.647088 4.785885 5.544406 4.785223 7.202977 2.952043
## [65] 6.926222 6.701133 3.745932 4.778343 5.758151 3.147744 4.416716 4.775059
## [73] 4.920073 2.994628 6.483927 6.180093 4.177351 5.027502 6.657769 2.484305
## [81] 4.397724 5.221102 5.882562 5.718818 3.006568 5.444817 5.906325 5.470833
## [89] 4.877094 3.246733 6.406116 5.479547 6.717818 5.932834 4.480049 2.402921
## [97] 7.095113 5.033035 3.589307 5.038582
```

Next, suppose the data and the covariates are available (practical scenario) and we want to draw inference about the regression coefficients.

Here, we stick to Bayesian estimation.

We use non-informative conjugate priors for the intercept $\beta_0 \sim \text{Normal}(0, 100^2)$ and variance $\sigma^2 \sim \text{Inverse-Gamma}(0.01, 0.01)$.

We will compare three different prior for the slopes $\beta_1, \dots, \beta_p$,

1. Noninformative Gaussian prior: $\beta_j \sim \text{Normal}(0, 100^2)$ IID over j
2. Gaussian shrinkage: $\beta_j \sim \text{Normal}(0, \sigma^2 \times \sigma_\beta^2)$ with $\sigma_\beta^2 \sim \text{Inverse-Gamma}(0.01, 0.01)$
3. Bayesian LASSO: $\beta_j \sim \text{Double-Exponential}(0, \sigma^2 \times \sigma_\beta^2)$ with $\sigma_\beta^2 \sim \text{Inverse-Gamma}(0.01, 0.01)$

Next, we fit the models using JAGS.

```
library(rjags)
```

```
## Loading required package: coda
```

```
## Linked to JAGS 4.3.0
```

```
## Loaded modules: basemod,bugs
```

```
data <- list(Y = Y, X = X, n = n, p = p)
params <- c("beta")
```

```
burn <- 10000
n.iter <- 20000
thin <- 10
n.chains <- 2
```

(1) Fit the noninformative Gaussian model

```
model_string <- textConnection("model{
# Likelihood
```

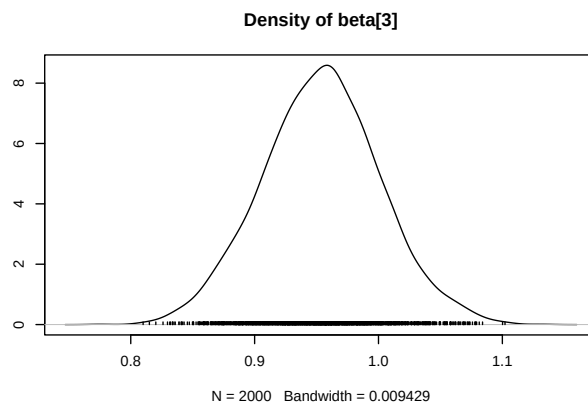
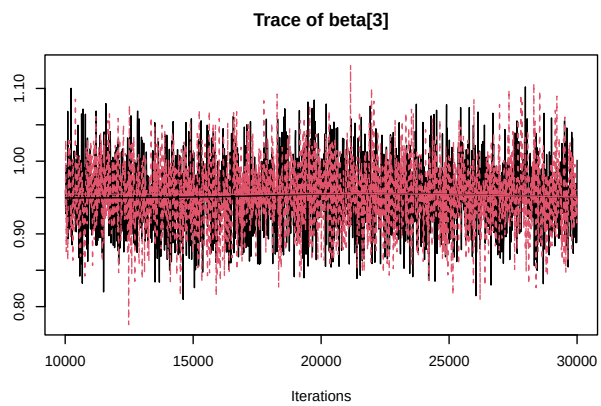
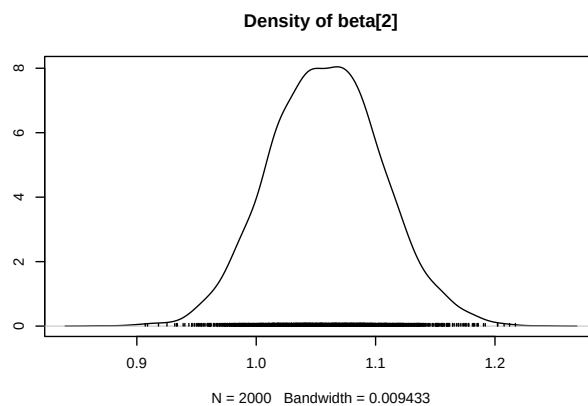
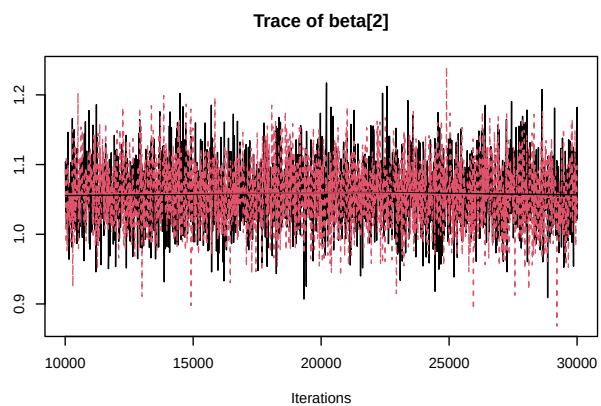
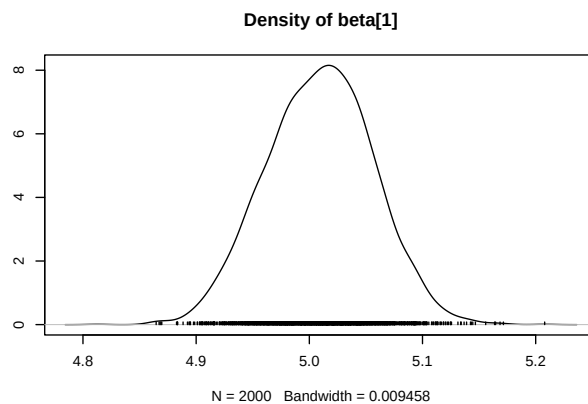
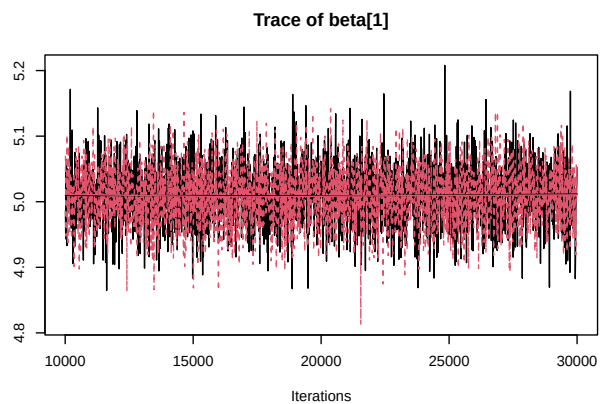
```

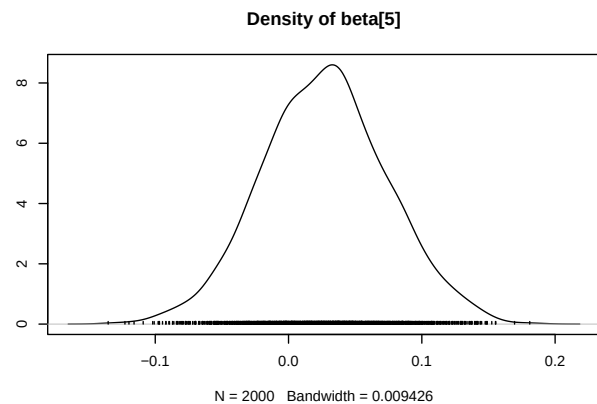
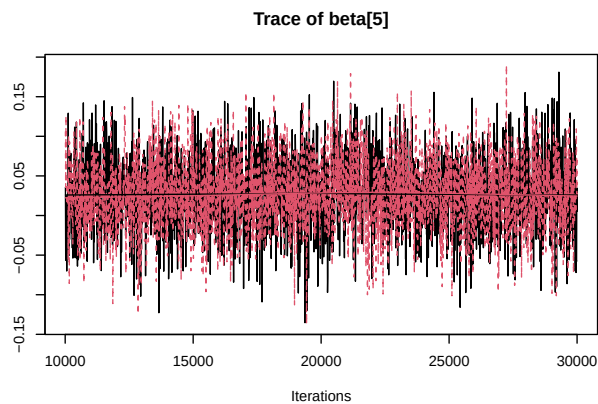
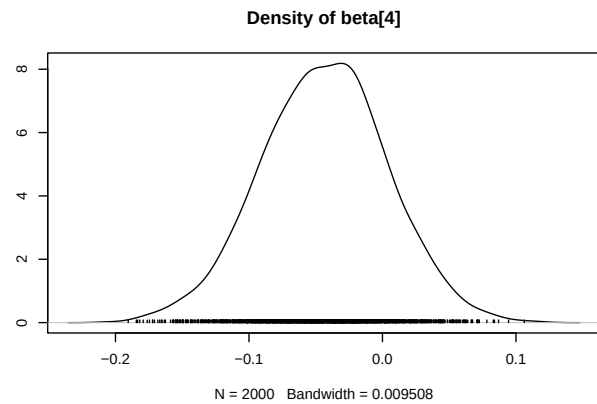
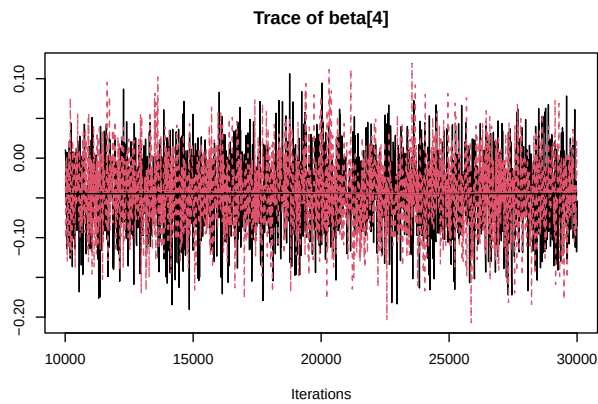
    for(i in 1:n){
      Y[i] ~ dnorm(inprod(X[i,],beta[]), tau)
    }
# Priors
beta[1] ~ dnorm(0, 0.0001)
for(j in 2:(p+1)){
  beta[j] ~ dnorm(0,0.0001)
}
tau ~ dgamma(0.01, 0.01)
}"))

model <- jags.model(model_string, data = data, n.chains = n.chains, quiet = TRUE)
update(model, burn, progress.bar = "none")
samples1 <- coda.samples(model, variable.names = params,
                        thin = thin, n.iter = n.iter, progress.bar = "none")

plot(samples1)

```





```
round(effectiveSize(samples1), 1)
```

```
## beta[1] beta[2] beta[3] beta[4] beta[5]
## 4000.0 4203.7 4000.0 4000.0 4000.0
```

```
summary(samples1)
```

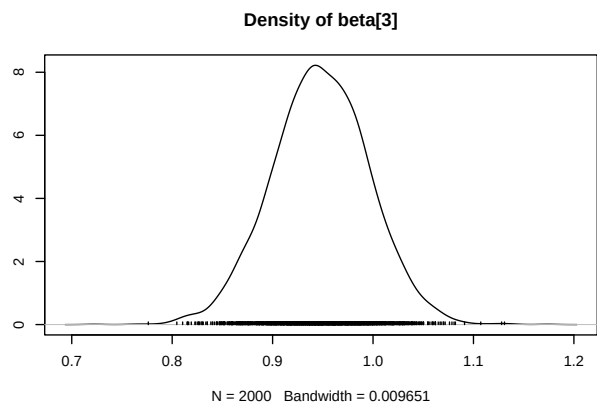
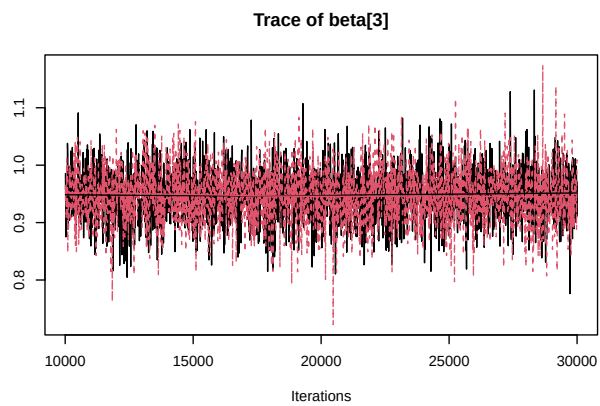
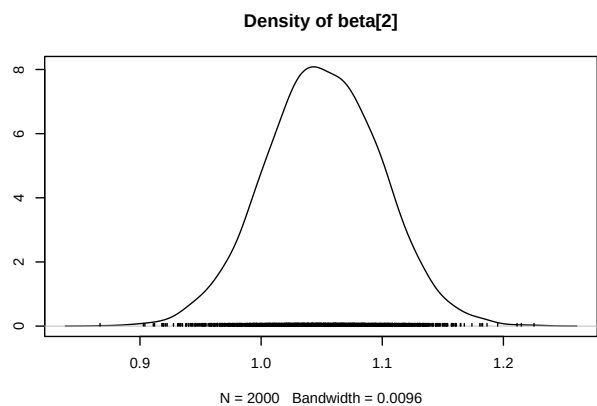
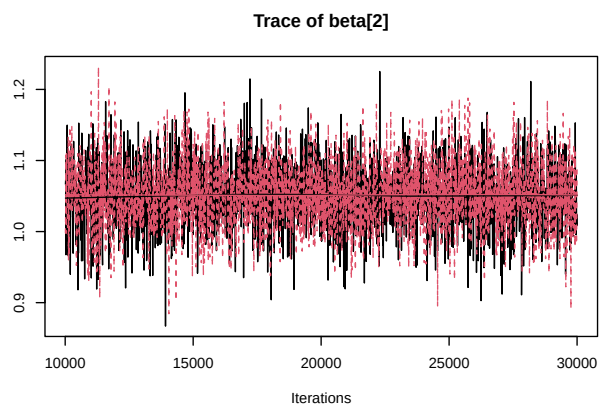
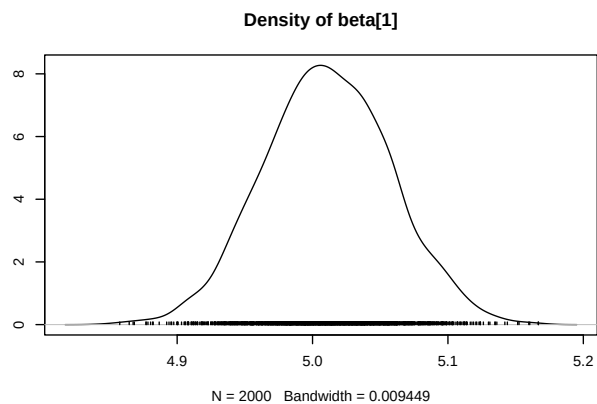
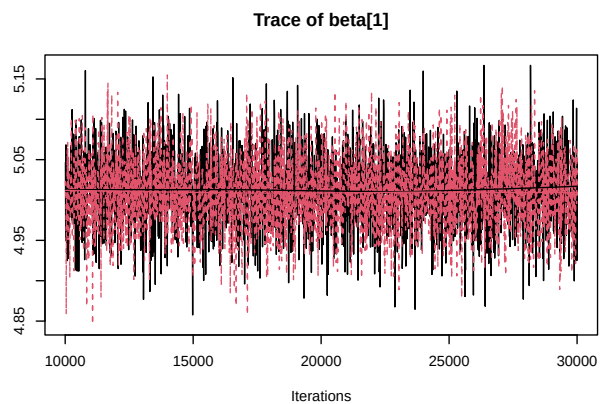
```
##
## Iterations = 10010:30000
## Thinning interval = 10
## Number of chains = 2
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## beta[1]  5.00915 0.04687 0.0007411      0.0007411
## beta[2]  1.05845 0.04675 0.0007391      0.0007225
## beta[3]  0.95364 0.04747 0.0007506      0.0007506
## beta[4] -0.04385 0.04716 0.0007457      0.0007456
## beta[5]  0.02696 0.04699 0.0007429      0.0007430
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## beta[1]  4.91815 4.977283 5.01030 5.04180 5.09799
## beta[2]  0.96929 1.025956 1.05831 1.08959 1.15278
```

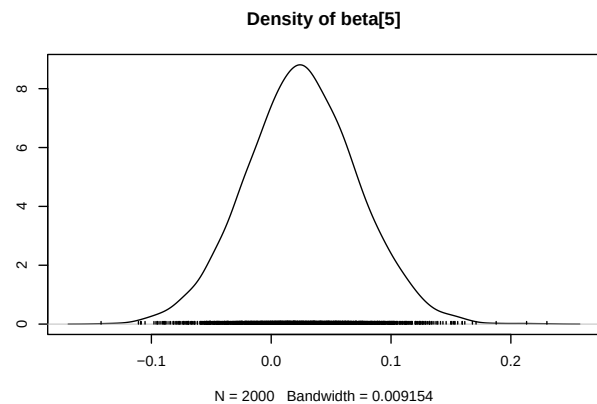
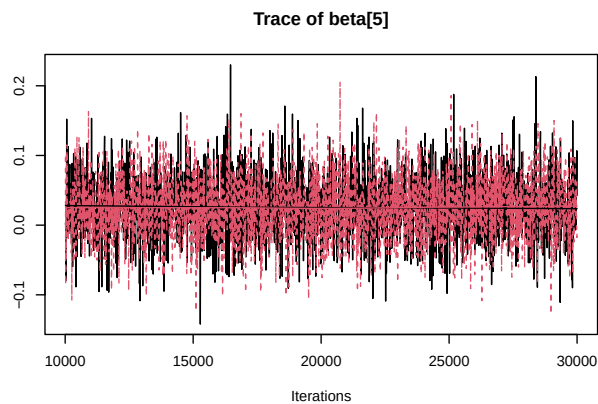
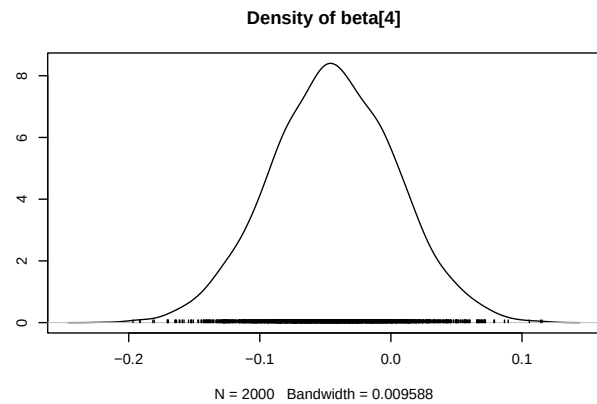
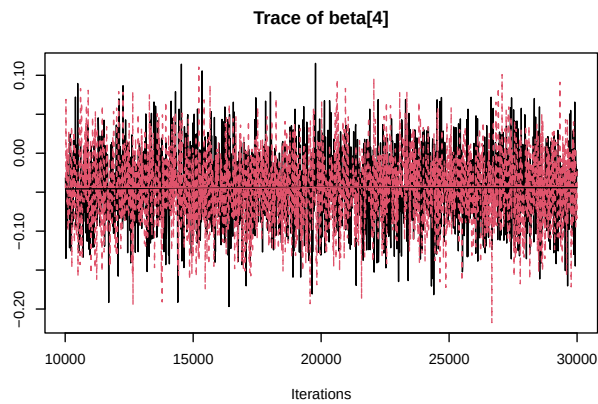
```
## beta[3]  0.86100  0.922321  0.95445  0.98494  1.05000
## beta[4] -0.13803 -0.075592 -0.04348 -0.01245  0.04688
## beta[5] -0.06463 -0.004787  0.02709  0.05781  0.12110
```

(2) Fit the Gaussian shrinkage model

```
model_string <- textConnection("model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(inprod(X[i,],beta[]), tau)
  }
  # Priors
  beta[1] ~ dnorm(0, 0.0001)
  for(j in 2:(p+1)){
    beta[j] ~ dnorm(0, tau * taub)
  }
  tau ~ dgamma(0.01, 0.01)
  taub ~ dgamma(0.01, 0.01)
}")

model <- jags.model(model_string, data = data, n.chains = n.chains, quiet = TRUE)
update(model, burn, progress.bar = "none")
samples2 <- coda.samples(model, variable.names = params,
                        thin = thin, n.iter = n.iter, progress.bar = "none")
plot(samples2)
```





```
round(effectiveSize(samples2), 1)
```

```
## beta[1] beta[2] beta[3] beta[4] beta[5]
## 3724.2 3783.9 4000.0 3616.2 3961.8
```

```
summary(samples2)
```

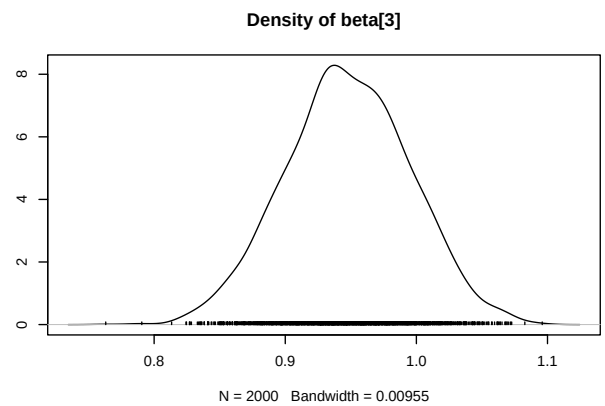
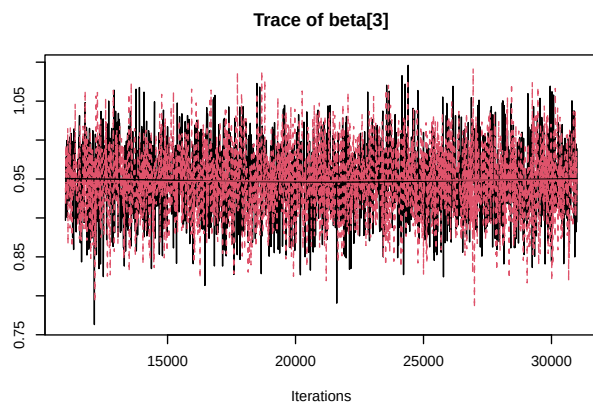
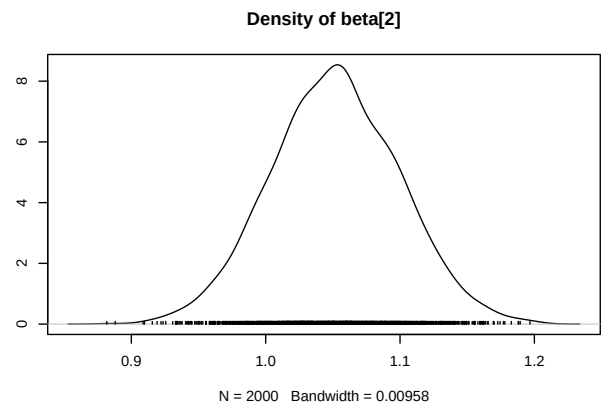
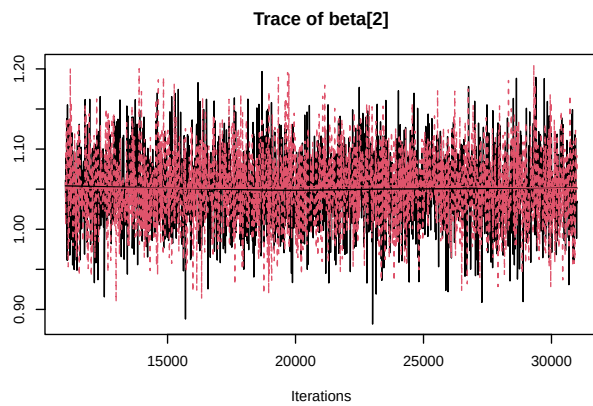
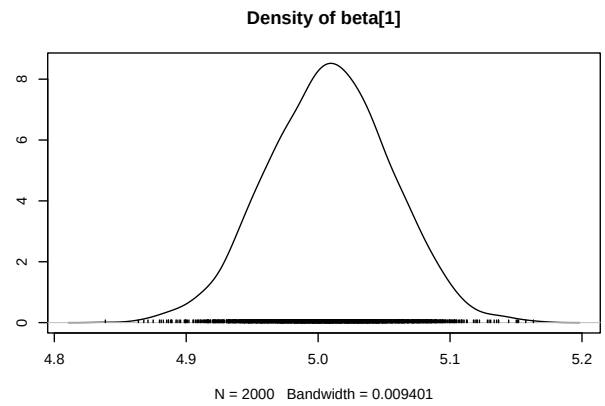
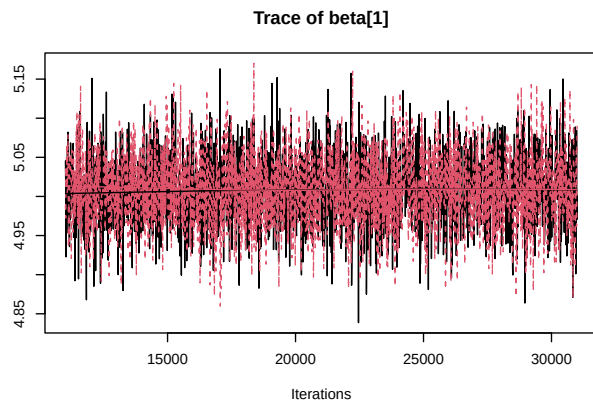
```
##
## Iterations = 10010:30000
## Thinning interval = 10
## Number of chains = 2
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## beta[1]  5.01096 0.04683 0.0007404      0.0007672
## beta[2]  1.05109 0.04757 0.0007522      0.0007753
## beta[3]  0.94746 0.04815 0.0007614      0.0007614
## beta[4] -0.04397 0.04751 0.0007513      0.0007930
## beta[5]  0.02574 0.04636 0.0007331      0.0007373
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%  97.5%
## beta[1]  4.91865 4.979338 5.01067 5.04275 5.1023
## beta[2]  0.95718 1.019441 1.05079 1.08366 1.1428
```

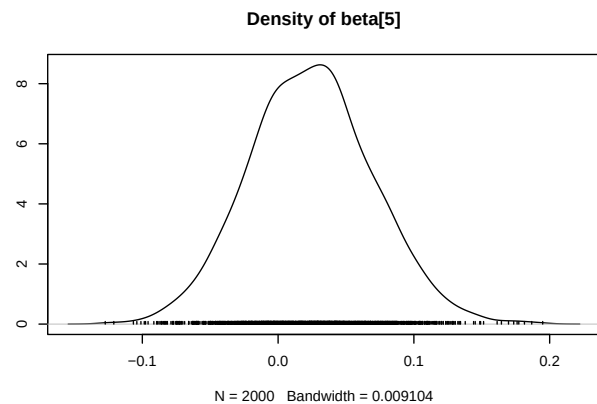
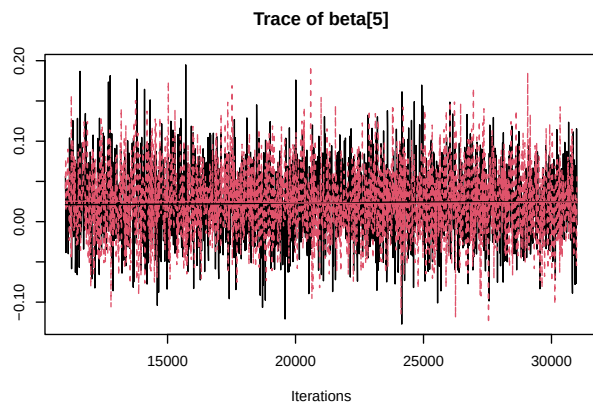
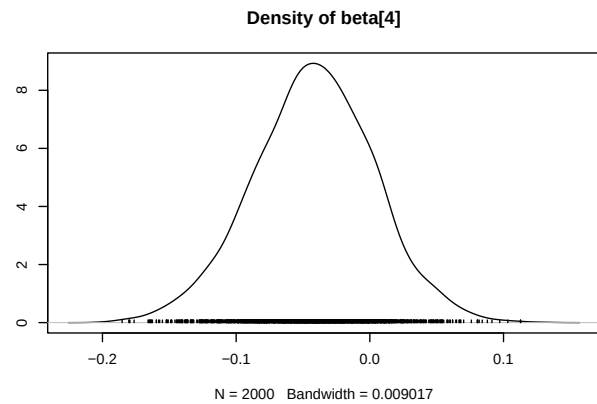
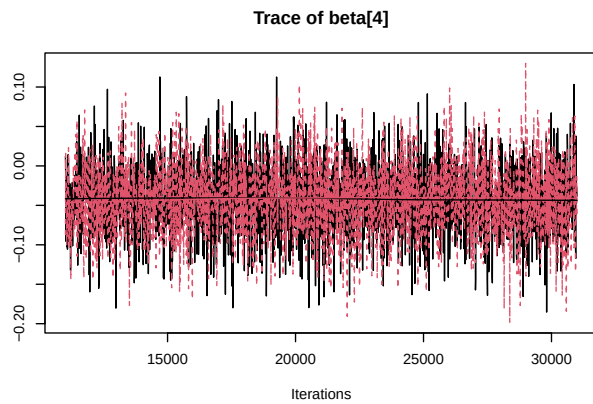
```
## beta[3]  0.85197  0.916114  0.94693  0.98020  1.0406
## beta[4] -0.13800 -0.075885 -0.04419 -0.01133  0.0501
## beta[5] -0.06603 -0.004737  0.02516  0.05605  0.1171
```

(3) Fit the Bayesian LASSO model

```
model_string <- textConnection("model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(inprod(X[i,],beta[]), tau)
  }
  # Priors
  beta[1] ~ dnorm(0, 0.0001)
  for(j in 2:(p+1)){
    beta[j] ~ ddexp(0, tau * taub)
  }
  tau ~ dgamma(0.01, 0.01)
  taub ~ dgamma(0.01, 0.01)
}")

model <- jags.model(model_string, data = data, n.chains = n.chains, quiet = TRUE)
update(model, burn, progress.bar = "none")
samples3 <- coda.samples(model, variable.names = params,
                          thin = thin, n.iter = n.iter, progress.bar = "none")
plot(samples3)
```





```
round(effectiveSize(samples3), 1)
```

```
## beta[1] beta[2] beta[3] beta[4] beta[5]
## 3868.7 4000.0 4000.0 4000.0 4328.6
```

```
summary(samples3)
```

```
##
## Iterations = 11010:31000
## Thinning interval = 10
## Number of chains = 2
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## beta[1]  5.00857 0.04659 0.0007366      0.0007492
## beta[2]  1.05159 0.04748 0.0007507      0.0007506
## beta[3]  0.94758 0.04733 0.0007483      0.0007484
## beta[4] -0.04198 0.04534 0.0007168      0.0007169
## beta[5]  0.02428 0.04562 0.0007214      0.0006957
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## beta[1]  4.91658 4.976541 5.00885 5.03986 5.09815
## beta[2]  0.95914 1.019597 1.05134 1.08422 1.14473
```

```
## beta[3]  0.85443  0.915913  0.94678  0.97994  1.03782
## beta[4] -0.13417 -0.071506 -0.04133 -0.01163  0.04751
## beta[5] -0.06228 -0.006803  0.02395  0.05365  0.11548
```

Compare the three fits

The plots below show the posterior for each covariate's slope for the three models:

Noninformative Gaussian is the black line Gaussian shrinkage is the red line Bayesian LASSO is the blue line

```
for(j in 2:(p+1)){

  # Collect the MCMC iteration from both chains for the three priors

  s1 <- c(samples1[[1]][,j],samples1[[2]][,j])
  s2 <- c(samples2[[1]][,j],samples2[[2]][,j])
  s3 <- c(samples3[[1]][,j],samples3[[2]][,j])

  # Get smooth density estimate for each prior

  d1 <- density(s1)
  d2 <- density(s2)
  d3 <- density(s3)

  # Plot the density estimates

  mx <- max(c(d1$y,d2$y,d3$y))

  plot(d1$x, d1$y, type = "l", ylim = c(0, mx),
       xlab = expression(beta), ylab = "Posterior density")
  lines(d2$x, d2$y, col = 2)
  lines(d3$x, d3$y, col = 4)
  abline(v = 0)
}
```

